

A SYNOPSIS ON

OS Optimizer Using AI

Submitted in partial fulfilment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

In

Computer Science & Engineering

Submitted by:

Niharika Pandey 2261391

Vijay Singh Mehta 2361007

Kirtika Joshi 2261323

Kartikey Bisht 2261315

Under the Guidance of

Mr. Ansh Dhingra

Lecturer

Project Team ID: 68



Department of Computer Science & Engineering

Graphic Era Hill University, Bhimtal, Uttarakhand

March-2025

CANDIDATE'S DECLARATION

We hereby certify that the work which is being presented in the Synopsis entitled “**OS Optimizer Using AI**” in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology in Computer Science & Engineering of the Graphic Era Hill University, Bhimtal campus and shall be carried out by the undersigned under the supervision of **Mr. Ansh Dhingra, Lecturer**, Department of Computer Science & Engineering, Graphic Era Hill University, Bhimtal.

Niharika Pandey 2261391

Vijay Singh Mehta 2361007

Kirtika Joshi 2261323

Kartikey Bisht 2261315

The above mentioned students shall be working under the supervision of the undersigned on the “**OS Optimizer using AI**”

Signature

Supervisor

Signature

Head of the Department

Internal Evaluation (By DPRC Committee)

Status of the Synopsis: Accepted / Rejected

Any Comments:

Name of the Committee Members:

Signature with Date

1.

2.

Table of Contents

Chapter No.	Description	Page No.
Chapter 1	Introduction and Problem Statement	1-2
Chapter 2	Background/ Literature Survey	3
Chapter 3	Objectives	4
Chapter 4	Hardware and Software Requirements	5
Chapter 5	Possible Approach/ Algorithms	6
	References	7

Chapter 1

Introduction and Problem Statement

The efficient allocation and supervision of organizational tasks within modern computing frameworks are critical to maximizing resource utilization and system performance. Traditional scheduling strategies, such as Round Robin, Shortest Job First (SJF), and Priority Scheduling, rely on predefined rules that fail to adapt to real-time system demands, leading to several performance bottlenecks, including:

1. **Resource Constraints:** Poor scheduling can overload certain system components while leaving others underutilized.
2. **Idle CPU Times:** Inadequate task prioritization can cause delays, leading to suboptimal CPU utilization.
3. **Increased Latency:** Inefficient scheduling can introduce lag, affecting the system's responsiveness.
4. **System Downtime and Failures:** The absence of a real-time monitoring mechanism makes it difficult to detect performance degradation or predict unexpected failures, increasing the risk of system crashes.

Proposed Solution

This work aims to develop an AI-powered Intelligent Task Scheduler and System Monitor that integrates Machine Learning (ML) and Reinforcement Learning (RL) to optimize system performance and resource utilization in real-time. Unlike traditional static scheduling algorithms (such as Round Robin, Shortest Job First (SJF), and Priority Scheduling), which operate based on fixed rules and predefined logic, this AI-driven approach will continuously learn and adapt to evolving workloads and system demands.

The proposed system will function as an autonomous decision-making engine that analyzes system parameters, predicts optimal scheduling actions, and dynamically adjusts resource allocation based on real-time conditions.

1) Machine Learning-Based Scheduling Optimization:

- I. Historical system performance data will be used to train supervised learning models that can predict the best task execution order.
- II. Feature engineering techniques will extract relevant parameters (CPU load, memory usage, process priority, etc.) to refine scheduling decisions.

2) Reinforcement Learning for Dynamic Resource Allocation:

- I. An RL - based agent will interact with the system environment, learning optimal strategies for CPU, memory, and I/O allocation through continuous feedback.
- II. Techniques like Q-Learning and Proximal Policy Optimization (PPO) will be explored to enhance decision-making in complex, multi-process environments.

3) Real-Time System Monitoring and Predictive Fault Detection:

- I. The system will employ anomaly detection models (e.g., Isolation Forest, LSTMs, and Autoencoders) to detect irregular patterns in system performance logs.
- II. Real-time alerts will be triggered for potential failures, allowing preventive maintenance and rapid troubleshooting.

By integrating Machine Learning, Reinforcement Learning, and Anomaly Detection, this project aims to create a self-optimizing, real-time task management and monitoring system that enhances efficiency, minimizes downtime, and ensures seamless computing operations in dynamic environments.

Chapter 2

Background

Traditional scheduling methods such as First-Come-First-Serve (FCFS), Shortest Job Next (SJN), and Multilevel Queue Scheduling rely on predefined static rules to allocate system resources. While these methods are simple to implement, they fail to adapt to real-time system dynamics, leading to inefficient resource utilization, higher latency, and increased system bottlenecks. Additionally, manual system monitoring is heavily dependent on user intervention, making it impractical and inefficient for large-scale computing environments where workloads fluctuate dynamically.

Limitations of Traditional Scheduling and Monitoring Approaches:

1. **Fixed Scheduling Rules:** Static algorithms do not consider real-time system loads, resulting in CPU underutilization or overload.
2. **Lack of Predictive Adaptation:** Traditional methods cannot predict system failures or optimize scheduling based on historical performance data.
3. **Manual Supervision:** Existing monitoring techniques require constant human oversight, making them unsuitable for autonomous, large-scale operations.

Advancements in AI-Driven Scheduling and Anomaly Detection

Several research studies have explored Machine Learning (ML)-based task scheduling and AI-powered anomaly detection to address these limitations:

ML-Based Scheduling Algorithms:

- I. Deep Q-Networks (DQN) and heuristic-based scheduling have demonstrated superior task prioritization by dynamically adapting to system conditions.
- II. Supervised learning models leverage historical system data to predict optimal scheduling decisions for maximizing efficiency.

Reinforcement Learning (RL) for Resource Management:

- I. RL techniques have been successfully applied in cloud computing environments to dynamically allocate resources based on workload demands.
- II. Methods such as Proximal Policy Optimization (PPO) and Q-learning enable intelligent, adaptive scheduling with continuous learning.

Anomaly Detection for System Failures:

- I. Isolation Forest, Autoencoders, and Long Short-Term Memory (LSTMs) have been widely used to detect unusual patterns in system performance logs.
- II. Real-time anomaly prediction allows for proactive failure mitigation, reducing unexpected downtimes.

Chapter 3

Objectives

The primary objectives of this project are:

1. AI-Powered Task Scheduling:

Develop an intelligent scheduler that dynamically prioritizes processes based on CPU load, memory usage, and real-time system conditions.

Use supervised learning to predict optimal scheduling decisions based on historical data.

2. Real-Time System Monitoring:

Implement a React.js dashboard to visualize system performance metrics such as CPU usage, memory utilization, disk I/O, and network activity.

Ensure continuous log tracking and event detection to capture system performance trends.

3. Anomaly Detection for Failure Prediction:

Use LSTMs, Isolation Forest, and Autoencoders to detect unexpected system behavior.

Generate real-time alerts for potential failures, security breaches, or performance degradation.

4. Reinforcement Learning for Resource Optimization:

Implement an RL-based model that continuously learns the best resource allocation strategies.

Optimize CPU, memory, and storage distribution to ensure efficient task execution and minimal wastage.

5. Deployment & Scalability:

Deploy the system on Heroku for real-world testing.

Ensure the system is scalable for enterprise-level environments, supporting multiple clients and tasks simultaneously

Chapter 4

Hardware and Software Requirements

4.1 Hardware Requirements

Sl. No	Name of the Hardware	Minimum Specification	Maximum Specification
1.	Processor	Intel I5/I7 or AMD equivalent	Intel I7/I9, Ryzen 7/9
2.	RAM	8 GB	16 GB or more
3.	Storage	50 GB Free Disk space	SSD with 100+ GB Free Space
4.	GPU	Not mandatory	NVIDIA CUDA-enabled GPU

4.2 Software Requirements

Sl. No	Component	Technology Used
1.	Frontend	HTML, CSS , Tailwind CSS, JavaScript, React.js
2.	Backend	Flask(Python), Restful API,SQL
3.	Machine learning	TensorFlow, Pytorch, Scikit-learn, Nump, Pandas, Matplotlib

Chapter 5

Possible Approach / Algorithms

1. Task Scheduling Algorithm

- 1) Traditional schedulers (Round Robin, Priority Scheduling, FCFS, etc.) will be compared with ML-based scheduling models.
- 2) Apply ML-driven task prioritization models, such as Supervised Learning and Deep Q-Networks (DQN), to predict optimal scheduling based on workload characteristics.
- 3) Implement Reinforcement Learning (RL) models, where an RL agent dynamically learns from system feedback and predicts the most efficient scheduling decisions.

2. Real-Time System Monitoring

- 1) Flask API will serve as the backend, collecting live system metrics such as CPU usage, memory utilization, disk activity, and network traffic using the psutil Python library.
- 2) A React.js-based front-end will display real-time system performance metrics.
- 3) D3.js will be used for dynamic, interactive visualizations, including CPU/RAM usage graphs, process monitoring, and anomaly alerts.

3. Anomaly Detection Algorithm

- 1) Isolation Forest, LSTMs, and Autoencoders will analyze system logs for unusual patterns.
- 2) Threshold-based anomaly alerts will notify users of potential failures.

4. Reinforcement Learning for Resource Allocation

- 1) An RL-based agent will learn from historical task execution patterns to improve resource scheduling decisions over time.
- 2) Proximal Policy Optimization (PPO) and Q-learning will be evaluated to determine the best strategy for real-time CPU and memory allocation.

5. Model Evaluation and Comparison

Various ML models will be benchmarked based on:

1. Prediction Accuracy (for anomaly detection): Confusion matrices, precision-recall curves, and F1-scores will measure anomaly detection accuracy.
2. Execution Speed (for real-time scheduling): Response time and scheduling latency will be compared across different scheduling algorithms.
3. System Efficiency (for resource allocation): The efficiency of CPU/memory utilization will be analyzed to determine the most effective RL-based resource allocation method.

References

[1]. "Machine Learning-Based Task Scheduling for Cloud Computing" – IEEE Xplore

DOI: 10.1109/TCC.2019.2939391

Discusses AI-driven dynamic scheduling techniques for cloud-based applications.

[2]. "AI-Based Task Scheduling and Load Balancing in Multi-Core Systems" – Springer Journal of Supercomputing

DOI: 10.1007/s11227-020-03342-9

Explores how AI can improve real-time scheduling efficiency in multi-core processors.

[3]. "An Overview of Reinforcement Learning in Resource Allocation" – Elsevier Future Generation Computer Systems

DOI: 10.1016/j.future.2021.07.005

Examines reinforcement learning models applied to system resource allocation.

[4]. "Reinforcement Learning: An Introduction" – Richard S. Sutton & Andrew G. Barto

A comprehensive guide to reinforcement learning, covering Q-learning, Deep Q-Networks (DQN), and Proximal Policy Optimization (PPO)

[5]. "Distributed and Cloud Computing: From Parallel Processing to the Internet of Things" – Kai Hwang, Geoffrey C. Fox, and Jack J. Dongarra

Covers advanced scheduling and resource management strategies for cloud-based systems. [6].

Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow – Aurélien Géron

Offers practical insights into deep learning models, anomaly detection, and ML-based optimizations.

[7]. TensorFlow Documentation – <https://www.tensorflow.org/>

Official documentation on deep learning models used for anomaly detection.

[8]. Scikit-Learn Documentation – <https://scikit-learn.org/stable/>

Resources on supervised and unsupervised learning algorithms.

[9]. PyTorch Documentation – <https://pytorch.org/>

Covers LSTM-based anomaly detection techniques for predictive system monitoring.

[10]. Flask for REST API Development – <https://flask.palletsprojects.com/>

Guides on developing backend APIs for system monitoring.